ZetaChain: Protocol Contracts Sui

Security Review

Solo review by:
Jayfromthe13th, Security Researcher

July 11, 2025

Contents

1 Introduction **2**
- 1.1 About Cantina . . . 2
- 1.2 Disclaimer . . . 2
- 1.3 Risk assessment . . . 2
  - 1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**

3 Dynamic Field Issues **4**

4 Custom Transfer Hooks **4**

5 Sui's Object Model vs. Traditional Nonce-Based Cross-Chain Messaging **4**

6 Event Ordering Vulnerability in Cross-Chain Message Processing **4**

7 Storage and Performance Considerations **5**

8 ID Leakage **5**

9 Other Brige Solutions **5**
- 9.1 Ika . . . 5
  - 9.1.1 Key Advantages . . . 6
  - 9.1.2 References . . . 6
- 9.2 Sui Bridge . . . 6
  - 9.2.1 Key Features . . . 6
  - 9.2.2 References . . . 6

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	*Must* fix as soon as possible (if already deployed).
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

ZetaChain is an L1 blockchain for chain abstraction, with the objective of building simple, secure, universal apps that span any chain from Ethereum and Cosmos to Bitcoin and beyond.

From Feb 11th to Feb 12th the security researchers conducted a review of protocol-contracts-sui on commit hash cb9c834f.

This document outlines important factors to consider when bridging assets to Sui via ZetaChain. It addresses potential challenges that can arise from differences in transaction models, token handling, and cross-chain message processing.

3 Dynamic Field Issues

Description: Some tokens may have dynamic fields (e.g., metadata or custom data) that require proper handling during transfers. If not managed correctly, these fields can cause runtime failures or unexpected behavior, especially with custom tokens. For example, transferring a token with dynamic fields out of a vault could lead to runtime errors if the fields are not serialized or deserialized properly. The gateway contract must validate token types, handle dynamic fields appropriately, and test extensively to ensure compatibility and prevent disruptions.

Recommendation: Overall, the team should be conscious of these types of tokens when Whitelisting tokens from Sui and consider adding in V2 validation for these token types, handle dynamic fields appropriately, and test extensively to ensure compatibility and prevent disruptions. Ensure dynamic fields are fully reconstructed before transferring tokens. Whitelist only tokens that do not introduce serialization/deserialization issues.

4 Custom Transfer Hooks

Description: Certain Sui tokens implement custom transfer logic (e.g., requiring an approval step, triggering callbacks, or enforcing specific conditions). If the gateway contract does not account for these behaviors, transfers could fail or revert, causing runtime issues when moving tokens into or out of vaults. For instance, a custom transfer hook might reject a transfer due to unsupported logic, leaving tokens stuck in the vault or causing inconsistent states.

Recommendation: Overall, the team should be conscious of these types of tokens as well. The Whitelist of compatible tokens is fine.If these tokens are considered then the contract should support custom transfer hooks.

5 Sui's Object Model vs. Traditional Nonce-Based Cross-Chain Messaging

Description: The gateway contract's current implementation attempts to apply traditional blockchain nonce-based sequencing to Sui's transaction model, creating potential security vulnerabilities. While traditional blockchains like Ethereum use account-based nonces for transaction ordering and replay protection, Sui employs a fundamentally different object-centric model with parallel transaction execution and checkpoint-based finality. This architectural mismatch poses serious risks to cross-chain message ordering and processing integrity.

In Sui's architecture, transactions can execute in parallel, and finality is determined by checkpoints rather than block confirmations. Traditional blockchains maintain a linear transaction history with predictable finality, whereas Sui's multi-lane processing and object-centric ownership model requires a different approach to transaction ordering and uniqueness verification.

Recommendation: The gateway contract should be redesigned to align with Sui's native characteristics rather than forcing traditional blockchain patterns. This includes adopting transaction digest-based identification, implementing checkpoint-aware message processing, and using causal ordering instead of sequential nonces. Bridge validators should wait for checkpoint finality, verify transaction inclusion proofs, and maintain digest-based transaction tracking. Key changes should include:

- Replace nonce-based sequencing with transaction digest tracking.
- Implement checkpoint-based finality verification.

6 Event Ordering Vulnerability in Cross-Chain Message Processing

Description: The gateway contract's reliance on Sui events for cross-chain messaging introduces ordering vulnerabilities. Unlike traditional blockchain events which maintain global ordering within blocks, Sui's event system does not guarantee consistent ordering across parallel transactions. This fundamental difference creates a significant risk when processing deposits on ZetaChain, as the sequence of operations may not match the original execution order on Sui.

The current implementation emits deposit events without sufficient ordering guarantees, assuming they will be processed in the same sequence they were created. However, Sui's parallel execution model means multiple deposits from the same sender can be processed simultaneously, and the events they generate may be observed in different orders by different validators. When these events are consumed by ZetaChain, there's no mechanism to ensure the preservation of the original transaction sequence, potentially leading to incorrect state transitions.

Recommendation: The cross-chain messaging system should be enhanced to include explicit ordering information and stronger consistency guarantees. Instead of relying solely on event emission order, the system should:

- Implement a robust message sequencing mechanism using Sui's checkpoint system (see the docs checkpoint verification section).
- Include causal ordering information in event payloads.
- Add sender-specific sequence numbers for related transactions.
- Utilize timestamp and checkpoint data for temporal ordering.
- Implement a message queue system on ZetaChain to enforce correct processing order.

7 Storage and Performance Considerations

Description: The current gateway implementation uses Sui's Bag for vault storage, which is efficient for frequent access patterns but operates within Sui's strict 256KB object size limit. As the number of token types and transaction volumes grow, the Bag could become bloated, leading to scalability issues, increased gas costs, and performance degradation. Unused or empty vaults remain in the Bag, consuming storage and increasing operational costs.

Recommendation: To address these issues, the gateway contract should:

- Implement a vault cleanup mechanism to remove unused or empty vaults.
- Monitor storage usage to prevent hitting Sui's 256KB object size limit.

8 ID Leakage

Description: The gateway contract may create or manage objects with unique IDs (e.g., vaults, capability objects). If IDs are reused or leaked, it can lead to security vulnerabilities, inconsistent state, or runtime errors. For example, reusing an existing object's ID when creating a new object can cause unauthorized access or data corruption. The Sui ID leak verifier ensures that only "*fresh*" IDs (generated by sui::object::new) are used and that IDs are not leaked through returns, mutable references, vectors, or function arguments.

Recommendation:

- Always generate fresh IDs using sui::object::new when creating new objects.
- Avoid reusing existing IDs or leaking them through returns, mutable references, vectors, or function arguments.
- Use Sui's ID leak verifier to check for potential ID leaks in the contract.

9 Other Brige Solutions

9.1 Ika

Cross-chain interoperability solution that enables trustless, native asset transfers without relying on centralized validators or wrapped tokens. Unlike traditional bridges, Ika leverages Zero Trust Architecture and 2PC-MPC cryptography, ensuring that transactions require both user and network signatures—eliminating single points of failure. Its dWallet system allows direct control of assets across multiple chains, avoiding the risks of wrapped tokens. Additionally, Ika's broadcast-based communication and Mysticeti consensus enable high scalability, supporting thousands of validators while maintaining efficiency.

9.1.1 Key Advantages

- Zero Trust Architecture – Eliminates reliance on validators by requiring both user and network signatures for transactions.
- True Native Asset Transfers – Directly moves BTC, ETH, and other native assets without using wrapped tokens.
- 2PC-MPC Cryptography – Enables ultra-fast (10,000 signatures/sec) and highly decentralized transaction processing.
- dWallet System – Allows programmable cross-chain asset control without lock-and-mint mechanisms.
- Broadcast-Based Communication – Uses Mysticeti consensus to scale efficiently with thousands of validators.
- High Scalability & Security – Maintains decentralization with a large validator set while ensuring efficient performance.
- Smart Contract-Controlled Security – Enables programmable, cryptographic enforcement of cross-chain asset movements.

9.1.2 References

- Ika Whitepaper.
- Ika Sui blog entry.

9.2 Sui Bridge

Sui Bridge is an official bridging solution that enables seamless transfers of assets like ETH, wBTC, USDC, and USDT between Ethereum and Sui. Unlike third-party bridges, Sui Bridge is designed to leverage Sui's native security and consensus mechanisms, ensuring fast and secure transactions. However, it still follows a more traditional bridging model, involving lock-and-mint mechanisms to represent assets on Sui.

9.2.1 Key Features

- Native Sui Integration – Designed specifically for bridging assets between Ethereum and Sui.
- Lock-and-Mint Model – Assets like ETH and USDC are locked on the source chain and minted as wrapped tokens on Sui.
- Validator & Smart Contract Security – Relies on validators and smart contracts to manage asset transfers.
- Simpler, Centralized Approach – More traditional bridging model compared to fully trustless solutions.
- Limited Scalability – Restricted by the architecture of its lock-and-mint mechanism.
- No Native Asset Transfers – Unlike Ika, does not support direct cross-chain movement of native assets.

9.2.2 References

- Zellic's Sui Bridge Audit.
- Sui Bridge Docs.